IC TRAINING CENTER VIET NAM
FUNDAMENTAL IC DESIGN AND VERIFICATION COURSE



FINAL PROJECT
TIMER IP DESIGN SPECIFICATION

Student : NGUYEN DUC ANH TUAN
Class : IC41
Instructor : TRAN MINH DUC

Contents

LIST OF TABLES **4**

LIST OF FIGURES **4**

1. Overview **5**

1.1. Introduction 5

1.2. Main features 5

1.3. Block diagram 6

1.4. Interface signals 7

2. Register Specification **8**

2.1. Register Summary 8

2.2. Timer Control Register (TCR) 9

2.3. Timer Data Register 0 (TDR0) 10

2.4. Timer Data Register 1 (TDR1) 11

2.5. Timer Compare Register 0 (TCMP0) 11

2.6. Timer Compare Register 1 (TCMP1) 12

2.7. Timer Interrupt Enable Register (TIER) 12

2.8. Timer Interrupt Status Register (TISR) 12

2.9. Timer Halt Control Status Register (THCSR) 13

3. Functional Description **14**

3.1. APB slave 14

3.2. Register Module 16

3.2.1. Write Path and Specific Register Implementations 17

3.2.2. Read Path and RAZ/WI Implementation 20

3.3. Counter Control 21
3.3.1. Logic Implementation 21
3.3.2. Timing and Behavior 23
3.4. Counter Module 23
3.4.1. Logic Implementation and Operation Priority 24
3.4.2. Advanced Features Timing 25
3.5. Interrupt Logic 26
3.5.1. Logic Implementation and Clear Priority 27
3.5.2. Interrupt Lifecycle Timing 28

LIST OF TABLES

Table 1.1: Interface signals of timer IP. 7
Table 2.1: Register Summary. 8
Table 2.2: Timer Control Register (TCR). 9
Table 2.3: Timer Data Register 0 (TDR0) 11
Table 2.4: Timer Data Register 1 (TDR1) 11
Table 2.5: Timer Compare Register 0 (TCMP0) 11
Table 2.6: Timer Compare Register 1 (TCMP1) 12
Table 2.7: Timer Interrupt Enable Register (TIER) 12
Table 2.8: Timer Interrupt Status Register (TISR) 13
Table 2.9: Timer Halt Control Status Register (THCSR) 13

LIST OF FIGURES

Figure 1.1: Block diagram of Timer IP. 7
Figure 3.1: APB Slave timing 15
Figure 3.2: APB slave logic diagram 16
Figure 3.3: TCR logic diagram and Error generation 17
Figure 3.4: TDR0 and TDR1 write path routing 18
Figure 3.5: TCMP register logic diagram demonstrating Byte Access 19
Figure 3.6: Interrupt Registers (TIER/TISR) and THCSR Logic Diagram. 20
Figure 3.7: Counter Control Logic Diagram 21
Figure 3.8: Counter Control Modes Timing 23
Figure 3.9: Counter Logic Diagram illustrating strict prioritization 24
Figure 3.10: Hardware Auto-Clear and Halted Mode timing 25
Figure 3.11: Interrupt Logic Diagram (Generation and RW1C Clearance) 27
Figure 3.12: Interrupt Generation and RW1C Clearance timing 28

1. Overview

1.1. Introduction

A timer is an essential peripheral module in any System-on-Chip (SoC) design, primarily utilized to generate accurate timing intervals, hardware delays, and event triggers. In this project, the Timer IP is customized and developed based on the Core-Local Interruptor (CLINT) module from the industrial RISC-V architecture. It functions as a highly configurable 64-bit counter that can generate maskable hardware level-interrupts when the running counter reaches a user-defined threshold. This makes the IP critical for operating system tick generation, precise task scheduling, and reliable hardware synchronization.

1.2. Main features

Bus Interface & System Parameters:

- The IP acts as an APB slave, configured via the APB bus with a 12-bit address space.
- Supports 32-bit APB transfers as well as byte access.
- Supports a 1-cycle wait state to improve timing.
- Supports error handling (error response) for prohibited accesses, such as writing a prohibited value to the divider or changing the div_en/div_val configuration while the timer is operating.
- Read/write accesses to reserved areas are handled as RAZ/WI (Read as Zero, Write Ignored).
- Uses an active-low asynchronous reset.

Counter Features:

- 64-bit count-up timer.
- Supports two counting modes: Default mode (counts at system clock speed) and Control mode (counting speed can be divided up to 256 based on the div_val register setting).
- The counter continues counting even when an interrupt or overflow occurs.

- Hardware automatically clears the counter registers (TDR0/TDR1) to their initial values when the timer enable signal (timer_en) transitions from High to Low (H->L). The timer resumes normal operation when timer_en goes High again.

Debug & Halted Mode:

- Supports halted mode (stop counting) when the system is in debug mode.
- The counter halts when both the input debug_mode signal is High and a halt request is issued (THCSR.halt_req = 1).
- The system provides a halt acknowledge flag (THCSR.halt_ack = 1) to indicate that the halt request is accepted. The counter resumes normal counting once the halt request is cleared to 0.

Interrupt Features:

- Generates a hardware, maskable, level-triggered interrupt.
- The timer interrupt signal (tim_int) is asserted when the interrupt is enabled and the counter value matches the 64-bit compare value.
- Once asserted, the interrupt remains pending and unchanged until it is explicitly cleared by software (by writing 1 to the TISR.int_st bit) or if the interrupt is disabled.

1.3. Block diagram

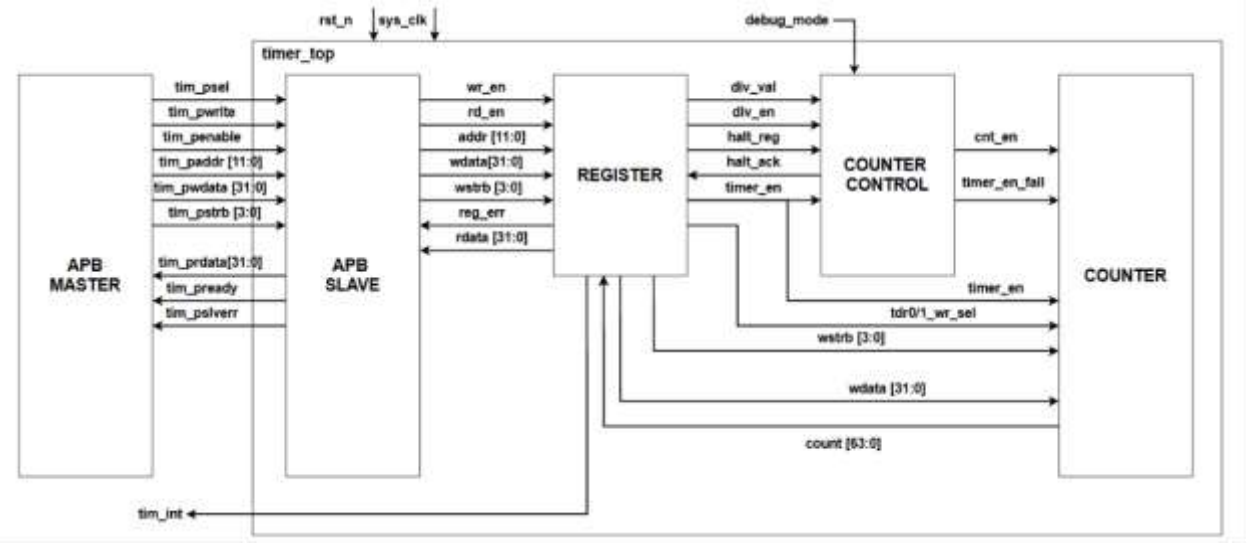


Figure 1.1: Block diagram of Timer IP.

Block Descriptions:

- **APB SLAVE:** This module acts as the communication bridge between the system's APB Master and the Timer IP. It handles the standard APB protocol handshaking, supporting 32-bit transfers, 1-cycle wait states, and byte-enable operations. It translates APB transactions into internal read/write strobes (wr_en, rd_en) for the Register module.
- **REGISTER:** This module functions as an address decoder and a static configuration holder. It stores system control settings (such as TCR, TIER, TISR, THCSR) and securely routes read/write data to the Counter block. Furthermore, it handles the RAZ/WI mechanism for reserved addresses and generates error responses (reg_err) for invalid user configurations.
- **COUNTER CONTROL:** This module is responsible for managing the counting speed and the halted state of the timer. By utilizing an internal prescaler (int_cnt), it evaluates the divisor configuration to generate the synchronized count enable pulse (cnt_en). It also strictly freezes the timer operations when a debug halt request is acknowledged.
- **COUNTER:** This is the core 64-bit execution module composed of the TDR0 and TDR1 registers. It strictly increments the timer value upon receiving the enable pulse, supports hardware auto-clearing via the timer_en_fall edge detector, allows direct byte-access overwrites from the APB bus, and encompasses the continuous comparator logic to assert the interrupt signal (tim_int).

1.4. Interface signals

The top module is named timer_top. The timer operates as an APB slave, and the ports are defined as follows:

Table 1.1: Interface signals of timer IP.

Signal Name	Width	Direction	Description
sys_clk	1	Input	System clock used for the timer operation.

sys_rst_n	1	Input	Active-low asynchronous reset.
tim_psel	1	Input	APB slave select signal.
tim_pwrite	1	Input	APB write enable signal. High for write, Low for read.
tim_penable	1	Input	APB enable signal.
tim_paddr[11:0]	12	Input	APB address bus (12-bit address space).
tim_pwdata[31:0]	32	Input	APB write data bus (32-bit transfer).
tim_prdata[31:0]	32	Output	APB read data bus.
tim_pstrb[3:0]	4	Input	APB write strobe signal (supports byte access).
tim_pready	1	Output	APB ready signal (supports wait state).
tim_pslverr	1	Output	APB slave error signal for error handling.
tim_int	1	Output	Timer interrupt signal.
dbg_mode	1	Input	Input debug mode signal. For simple, testbench should not change dbg_mode after timer_en is High (during timer operation).

2. Register Specification

2.1. Register Summary

Table 2.1: Register Summary.

Address	Abbreviation	Register name
0x00	TCR	Timer Control Register
0x04	TDR0	Timer Data Register 0
0x08	TDR1	Timer Data Register 1
0x0C	TCMP0	Timer Compare Register 0
0x10	TCMP1	Timer Compare Register 1
0x14	TIER	Timer Interrupt Enable Register

0x18	TISR	Timer Interrupt Status Register
0x1C	THCSR	Timer Halt Control Status Register
Others	Reserved	

2.2. Timer Control Register (TCR)

- **Offset address:** 0x00
- **Reset value:** 0x0000_0100

Table 2.2: Timer Control Register (TCR).

Bit	Name	Type	Default value	Description
31:12	Reserved	RO	20'h0	Reserved. Read as zero, write ignored (RAZ/WI).
11:8	DIV_VAL[3:0]	RW	4'b0001	**Counter control mode setting:** 4'b0000: Counting speed is not divided. 4'b0001: Counting speed is divided by 2 (default). 4'b0010: Counting speed is divided by 4. 4'b0011: Counting speed is divided by 8. 4'b0100: Counting speed is divided by 16. 4'b0101: Counting speed is divided by 32. 4'b0110: Counting speed is divided by 64. 4'b0111: Counting speed is divided by 128. 4'b1000: Counting speed is divided by 256.

				Others: Reserved (prohibited settings). **Note:** Users must not change div_val while timer_en is High. Accessing this field with a prohibited value or while the timer is operating will trigger an error response, and the register value remains unchanged.
7:2	Reserved	RO	6'b0	Reserved. Read as zero, write ignored (RAZ/WI).
1	DIV_EN	RW	1'b0	**Counter control mode enable:** 0: Disabled. Counter counts with normal speed based on system clock. 1: Enabled. The counting speed of the counter is controlled based on div_val. **Note:** Users must not change div_en while timer_en is High. Accessing this field while the timer is operating will trigger an error response.
0	TIM_EN	RW	1'b0	**Timer enable:** 0: Disabled. Counter does not count. 1: Enabled. Counter starts counting. **Note:** When timer_en changes from High to Low (H->L), it automatically initializes the TDR0/1 registers to their initial values via hardware.

2.3. Timer Data Register 0 (TDR0)

- **Offset address:** 0x04
- **Reset value:** 0x0000_0000

Table 2.3: Timer Data Register 0 (TDR0)

Bit	Name	Type	Default value	Description
31:0	TDR0	RW	32'h0000_0000	Lower 32-bit of the 64-bit counter. **Note:** The value of this register is cleared to its initial value (32'h0000_0000) purely by hardware when timer_en transitions from High to Low (H->L).

2.4. Timer Data Register 1 (TDR1)

- **Offset address:** 0x08
- **Reset value:** 0x0000_0000

Table 2.4: Timer Data Register 1 (TDR1)

Bit	Name	Type	Default value	Description
31:0	TDR1	RW	32'h0000_0000	Upper 32-bit of the 64-bit counter. **Note:** The value of this register is cleared to its initial value (32'h0000_0000) purely by hardware when timer_en transitions from High to Low (H->L).

2.5. Timer Compare Register 0 (TCMP0)

- **Offset address:** 0x0C
- **Reset value:** 0xFFFF_FFFF

Table 2.5: Timer Compare Register 0 (TCMP0)

Bit	Name	Type	Default value	Description
31:0	TCMP0	RW	32'hFFFF_FFFF	Lower 32-bit of the 64-bit compare value. The interrupt pending bit is asserted when the 64-bit counter value is exactly equal to the 64-bit compare value.

2.6. Timer Compare Register 1 (TCMP1)

- **Offset address:** 0x10
- **Reset value:** 0xFFFF_FFFF

Table 2.6: Timer Compare Register 1 (TCMP1)

Bit	Name	Type	Default value	Description
31:0	TCMP1	RW	32'hFFFF_FFFF	Upper 32-bit of the 64-bit compare value. The interrupt pending bit is asserted when the 64-bit counter value is exactly equal to the 64-bit compare value.

2.7. Timer Interrupt Enable Register (TIER)

- **Offset address:** 0x14
- **Reset value:** 0x0000_0000

Table 2.7: Timer Interrupt Enable Register (TIER)

Bit	Name	Type	Default value	Description
31:1	Reserved	RO	31'h0	Reserved. Read as zero, write ignored (RAZ/WI).
0	INT_EN	RW	1'b0	**Timer interrupt enable:** 0: Timer interrupt is disabled. No timer interrupt is outputted. 1: Timer interrupt is enabled. The timer interrupt pending bit can be outputted. **Note:** Clearing this bit to 0 while the interrupt is asserting will mask the external interrupt output to 0 but does not affect the internal interrupt pending bit (TISR.int_st).

2.8. Timer Interrupt Status Register (TISR)

- **Offset address:** 0x18

- **Reset value:** 0x0000_0000

Table 2.8: Timer Interrupt Status Register (TISR)

Bit	Name	Type	Default value	Description
31:1	Reserved	RO	31'h0	Reserved. Read as zero, write ignored (RAZ/WI).
0	INT_ST	RW1C	1'b0	**Timer interrupt trigger condition status bit (pending bit):** 0: The interrupt trigger condition does not occur. 1: The interrupt trigger condition occurred. **Clearance:** Write 1 when this bit is 1 to clear it to 0. Writing 0 when this bit is 1 has no effect. Writing to this bit when it is 0 has no effect. **Note 1:** When the interrupt trigger condition occurs (counter reaches compare value), the counter continues to count normally. **Note 2:** The clearance condition (writing 1) has the highest priority.

2.9. Timer Halt Control Status Register (THCSR)

- **Offset address:** 0x1C
- **Reset value:** 0x0000_0000

Table 2.9: Timer Halt Control Status Register (THCSR)

Bit	Name	Type	Default value	Description
31:2	Reserved	RO	30'h0	Reserved. Read as zero, write ignored (RAZ/WI).
1	HALT_ACK	RO	1'b0	**Timer halt acknowledge:** 0: Timer is NOT halted. 1: Timer is halted.

				The timer accepts the halt request only in debug mode, which is indicated by the external debug_mode input signal.
0	HALT_REQ	RW	1'b0	Timer halt request 0: no halt req. 1: timer is requested to halt.

3. Functional Description

3.1. APB slave

The Timer IP integrates an APB Slave module to communicate with the system's APB Master. This module is responsible for decoding APB bus transactions and generating corresponding internal control signals (wr_en, rd_en) for the Register module. The APB slave interface has the following key features:

- **Address and Data Width:** Operates with a 12-bit address bus (tim_paddr[11:0]) and 32-bit read/write data buses (tim_pwdata, tim_prdata).
- **Byte Access:** Supports writing to individual bytes using the 4-bit write strobe signal (tim_pstrb[3:0]).
- **Wait State Generation:** Implements a 1-cycle wait state by delaying the tim_pready signal to improve timing paths for internal register accesses.
- **Error Handling:** Generates an error response via the tim_pslverr signal if the master attempts an illegal operation (writing prohibited values or modifying configuration bits while the timer is actively counting).

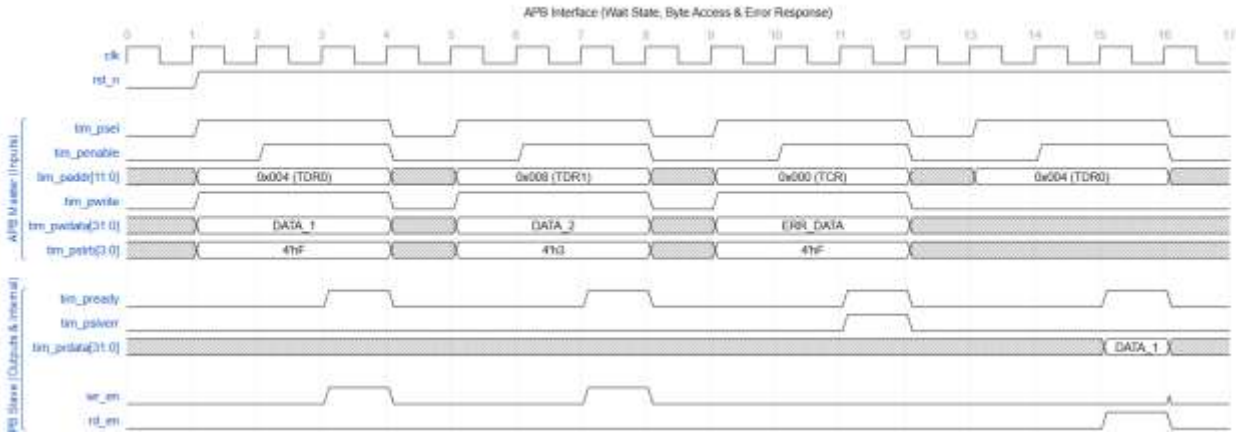


Figure 3.1: APB Slave timing

Waveform Analysis: As illustrated in Figure 3.1, the APB interface requires a 1-cycle wait state for every transaction.

During a valid write operation, tim_pready is asserted one clock cycle after tim_penable goes High. Simultaneously, the internal wr_en pulse is generated to allow the Register module to capture tim_pwdata and tim_pstrb.

If an illegal access is detected by the Register module (configuring div_val while timer_en is High), the internal reg_err signal is asserted. This forces tim_pslverr to High on the APB bus during the active phase of tim_pready, and the wr_en signal is masked to prevent invalid data from corrupting the registers.

For read operations, tim_pready and rd_en are similarly delayed by one cycle, ensuring the Register module has sufficient time to multiplex the correct data onto the tim_prdata bus.

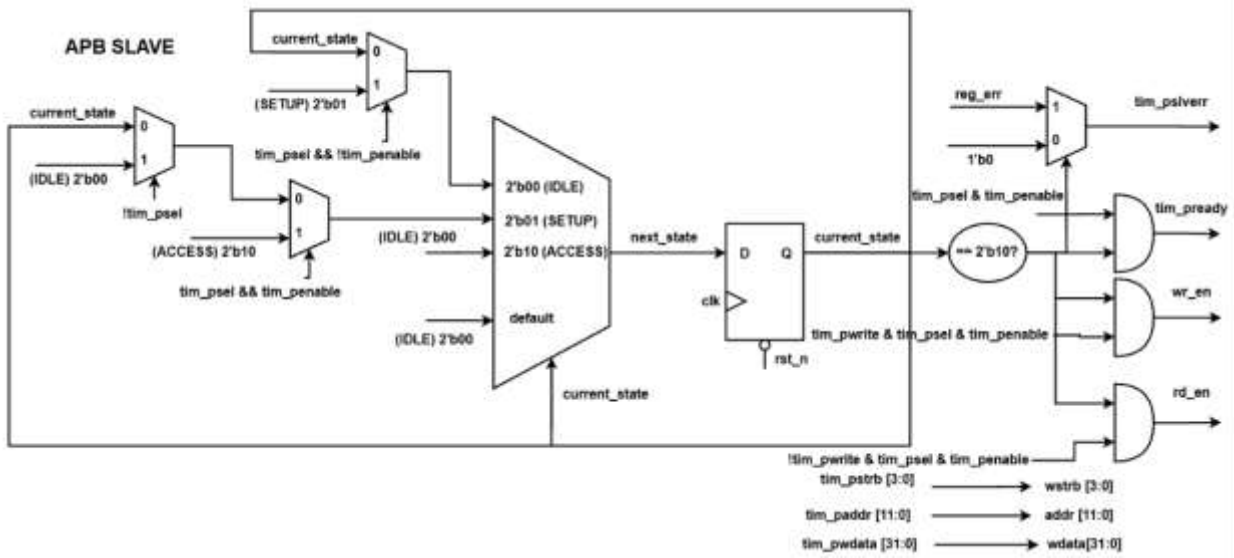


Figure 1.2: APB slave logic diagram

Logic Diagram Explanation:

The APB slave is a 3-state FSM (IDLE -> SETUP -> ACCESS) that converts APB transfers into internal control signals for the Register module.

- IDLE: waits for PSEL = 1 and PENABLE = 0, then moves to SETUP.
- SETUP: moves to ACCESS when PSEL = 1 and PENABLE = 1 (the wait cycle). Goes back to IDLE if PSEL is dropped.
- ACCESS: lasts exactly one cycle, then returns to IDLE.

Every transfer therefore has one wait state: PREADY is Low in the first PENABLE cycle and High in the next one. In the ACCESS state with PSEL = 1 and PENABLE = 1:

- tim_pready is asserted.
- wr_en (PWRITE = 1) or rd_en (PWRITE = 0) is asserted for one cycle.
- tim_pslverr follows reg_err from the Register module.

3.2. Register Module

The Register module functions as the central configuration hub of the Timer IP. Its primary responsibilities include decoding APB addresses to route write data to specific

internal registers, handling the byte-access logic, multiplexing read data back to the APB bus, and strictly managing error conditions for invalid configurations.

3.2.1. Write Path and Specific Register Implementations

Upon receiving the wr_en pulse and addr[11:0] from the APB Slave interface, the Register module generates exclusive write-select signals for individual registers. The implementation varies depending on the functional requirements of each register group:

A. Timer Control Register (TCR) and Error Handling

The TCR configures the fundamental behaviors of the timer, including the operational mode (div_en), divisor value (div_val), and overall enable state (timer_en). Because modifying these parameters while the timer is actively running can cause unpredictable counting behavior, the TCR incorporates strict hardware protection logic.

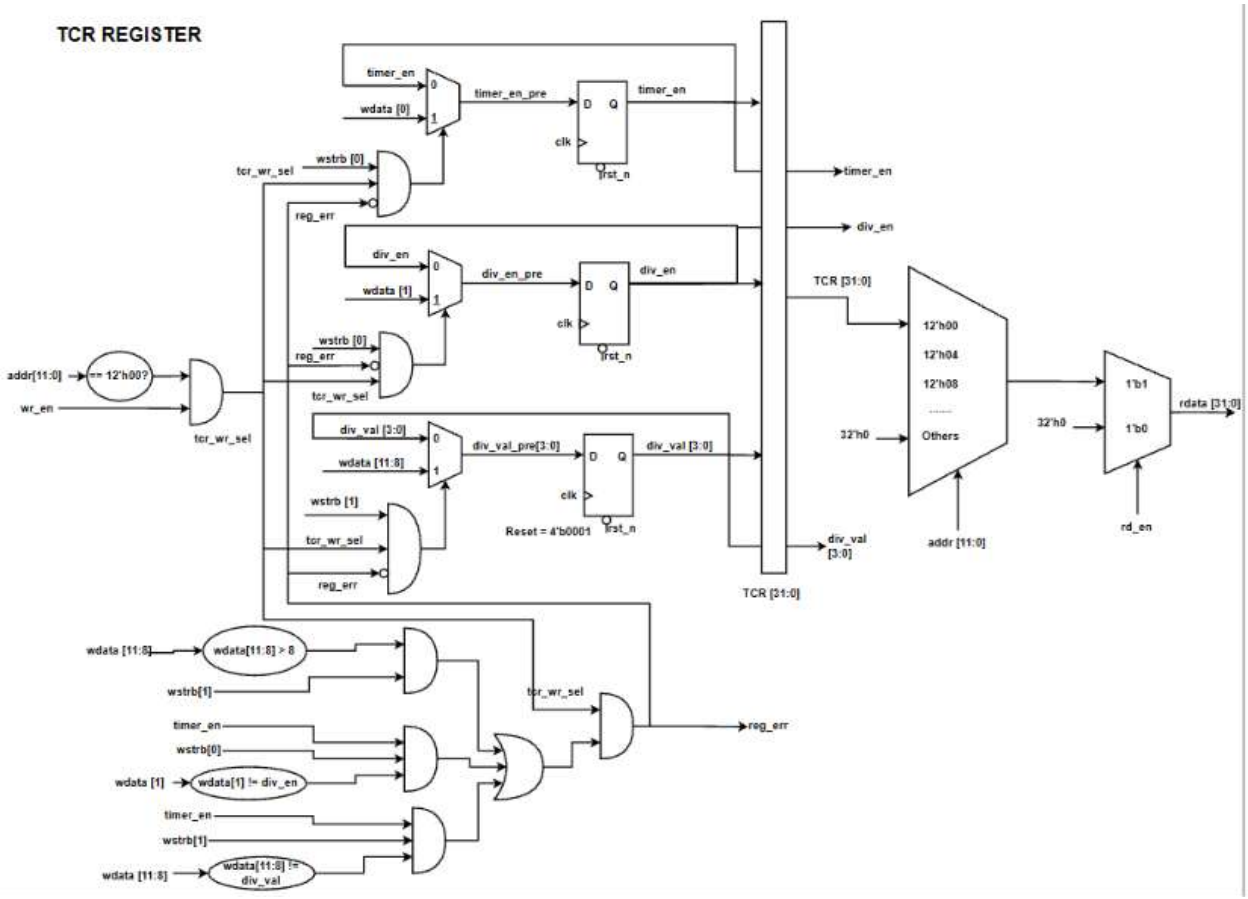


Figure 3.3: TCR logic diagram and Error generation

As illustrated in Figure 3.3, a combinatorial logic block continuously monitors the APB write data (wdata) alongside the current TCR state. The internal reg_err signal is asserted if a write operation attempts to:

1. Write a prohibited value to div_val (wdata[11:8] > 4'h8).
2. Change div_val while timer_en is currently High (timer_en == 1 and wdata[11:8] != div_val).
3. Change div_en while timer_en is currently High (timer_en == 1 and wdata[1] != div_en).

When reg_err is High, it acts as an active-low mask (via an AND gate) for the tcr_wr_sel signal, blocking the write operation to the internal D-Flip-Flops and preserving the previous safe values. The reg_err is then forwarded to the APB Slave to trigger the tim_pslverr response.

B. Timer Data Registers (TDR0, TDR1)

Unlike static configuration registers, TDR0 and TDR1 are dynamic counting registers physically located inside the Counter Module.

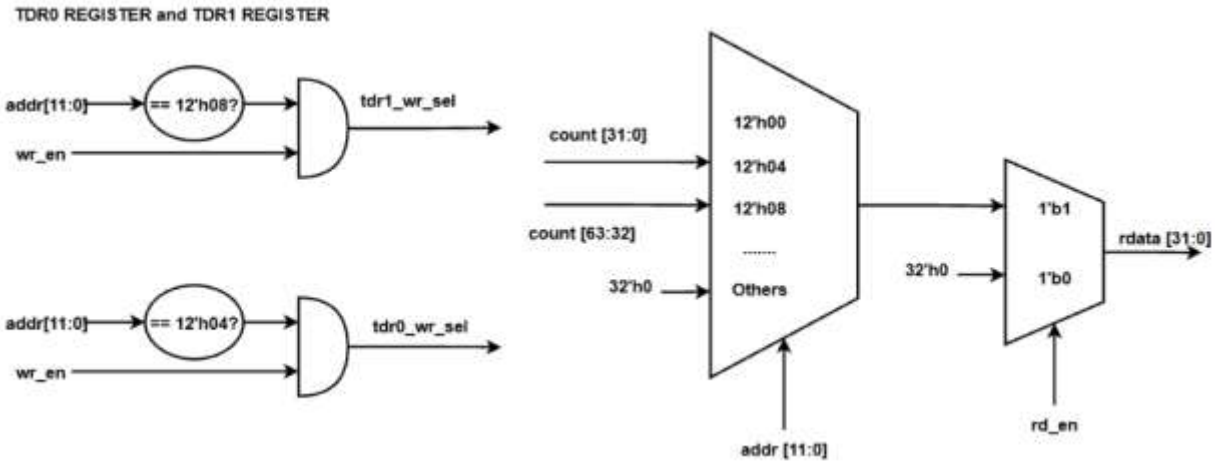


Figure 3.4: TDR0 and TDR1 write path routing

As shown in Figure 3.4, the Register module simply decodes the addresses (12'h04 for TDR0, 12'h08 for TDR1) and performs an AND operation with wr_en to generate tdr0_wr_sel and tdr1_wr_sel. These control signals, along with the unaltered wdata[31:0]

and wstrb[3:0], are routed directly to the Counter Module, delegating the actual byte-enable write overrides to the counter's internal multiplexers.

C. Timer Compare Registers (TCMP0, TCMP1)

TCMP0 and TCMP1 store the 64-bit threshold value used for generating interrupts. They fully support the Advanced Level's byte-access requirement.

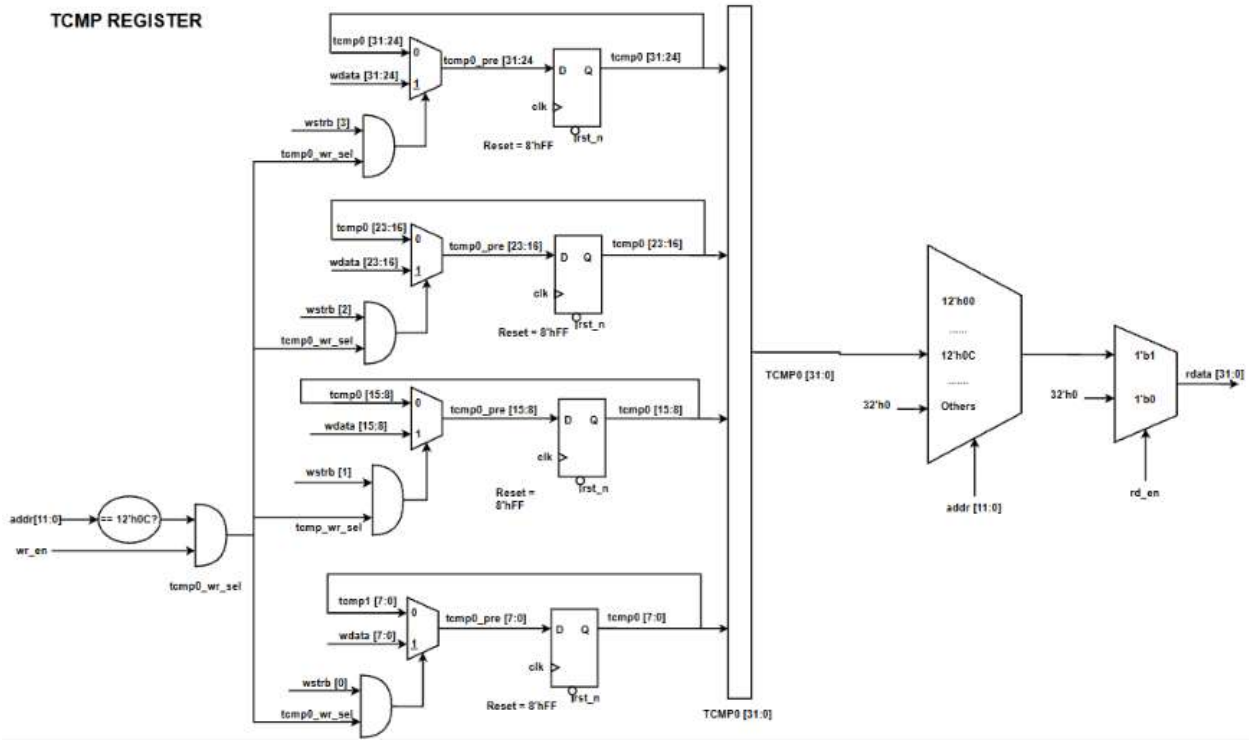


Figure 3.5: TCMP register logic diagram demonstrating Byte Access

Figure 3.5 demonstrates the standard byte-access architecture used for TCMP0 (and similarly for TCMP1, TIER, and THCSR). The 32-bit register is divided into four 8-bit D-Flip-Flop groups. The global tcmp0_wr_sel is AND-ed independently with each bit of the wstrb[3:0] signal. Consequently, an 8-bit block will only latch new data from the wdata bus if its corresponding write strobe bit is High; otherwise, the internal feedback loop maintains the current value.

D. Interrupt and Halt Registers (TIER, TISR, THCSR)

- **TIER (0x14):** Utilizes standard byte-access logic to store the int_en bit, enabling or masking the external interrupt output.

- **TISR (0x18):** This status register incorporates a Write-1-to-Clear (RW1C) mechanism. Instead of directly loading data from the bus, writing a 1 to wdata[0] generates an internal int_clear pulse, which resets the interrupt pending Flip-Flop back to 0.
- **THCSR (0x1C):** Stores the halt_req bit requested by the user and reflects the halt_ack status generated by the Counter Control module.

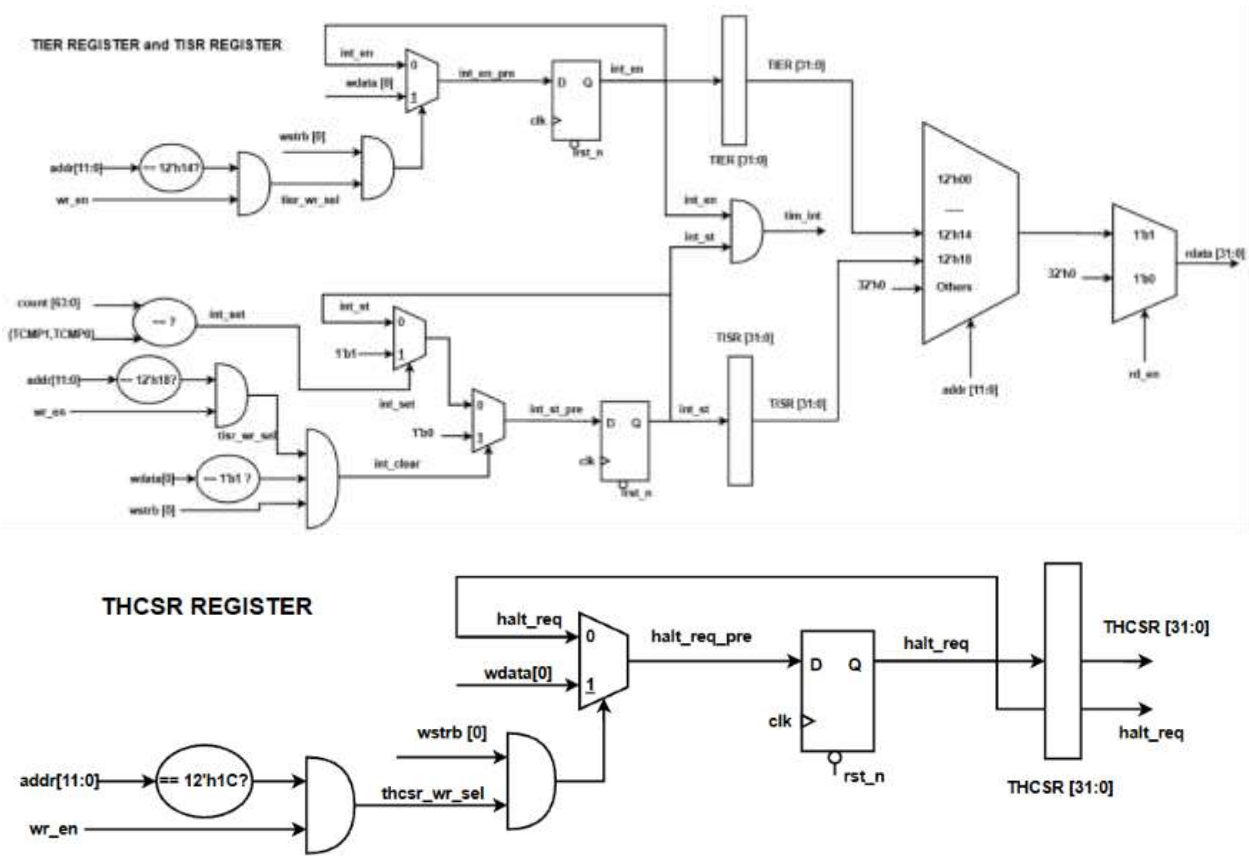


Figure 3.6: Interrupt Registers (TIER/TISR) and THCSR Logic Diagram.

3.2.2. Read Path and RAZ/WI Implementation

The read path is unified across all registers using a large multiplexer (Read MUX) structure. When the APB master initiates a read transfer, the addr[11:0] signal acts as the selector for the Read MUX.

- For internally stored registers (TCR, TCMP0/1, TIER, TISR, THCSR), the MUX selects the Q-outputs of their respective Flip-Flops.
- For the dynamic counting values (TDR0, TDR1), the MUX selects the live count[31:0] and count[63:32] buses fed back from the Counter Module.
- **RAZ/WI Compliance:** If the address does not match any valid register offset, the MUX defaults to the "Others" branch, which is hardwired to 32'h0000_0000.
- Finally, the output of the Read MUX passes through an isolation MUX controlled by rd_en. If rd_en is Low, the tim_prdata bus is forced to 0, ensuring that residual data is not leaked onto the APB bus outside of active read phases.

3.3. Counter Control

The Counter Control module serves as the core timing manager of the Timer IP. It evaluates user configurations (div_en, div_val) to generate a strictly synchronized count enable pulse (cnt_en) for the main 64-bit counter. Furthermore, it manages the debug halt mechanism, ensuring the timer operations can be frozen without losing counting precision.

3.3.1. Logic Implementation

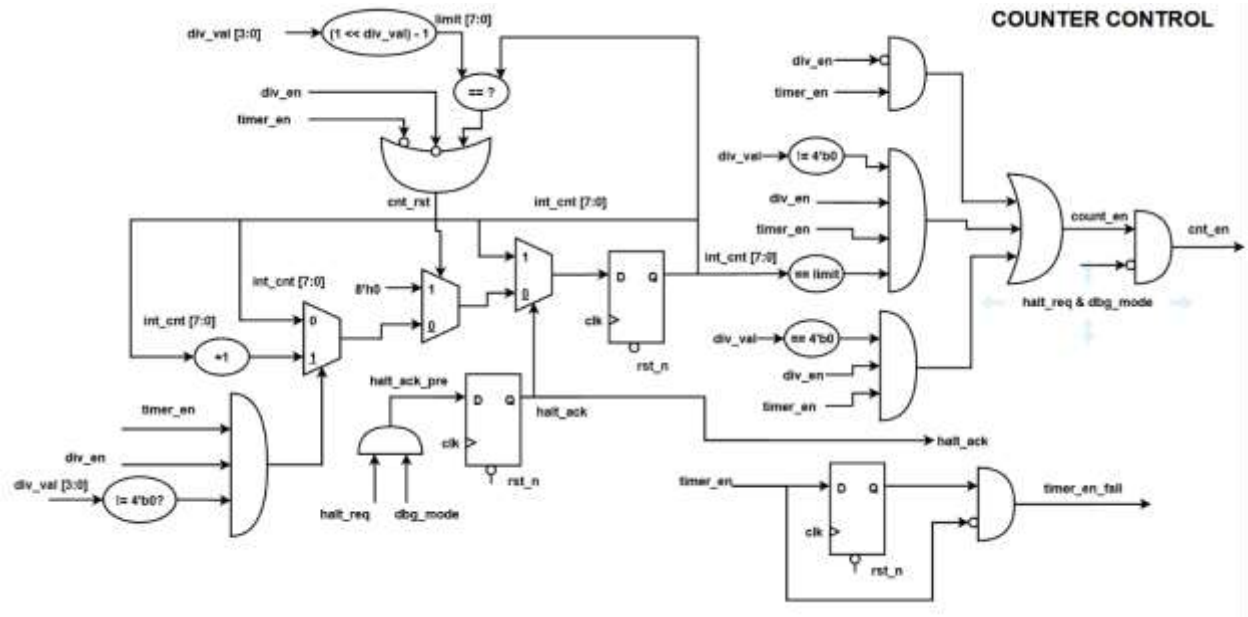


Figure 3.7: Counter Control Logic Diagram

As detailed in Figure 3.7, the logic revolves around an 8-bit internal prescaler counter (int_cnt) and several combinatorial evaluation paths:

- **Limit Calculation:** Instead of using physical clock dividers, the IP uses an enable-pulse generation technique to remain in a single clock domain. The maximum counting limit for the prescaler is calculated dynamically using a left-shift operator: limit = (1 << div_val) - 1.
- **Prescaler Reset Logic (cnt_rst):** The internal counter must reset to 8'h00 if the timer is disabled (timer_en = 0), if the control mode is disabled (div_en = 0), or if it has reached its counting threshold (int_cnt == limit). A NOR-based logic gates these conditions to assert the cnt_rst signal, controlling the synchronous clear multiplexer.
- **Count Enable (cnt_en) Generation:** The preliminary enable signal (count_en) is asserted under three valid configurations, handled by a 3-input OR gate:
    1. Default Mode: ~div_en & timer_en. (The counter increments on every system clock cycle).
    2. Control Mode (Un-divided): div_en & timer_en & (div_val == 0).
    3. Control Mode (Divided): div_en & timer_en & (div_val != 0) & (int_cnt == limit).
- **Halted Logic:** The halt_ack flag is asserted synchronously 1 clock cycle after the halt request condition is met. This flag switches a multiplexer to feed the current int_cnt value directly back into its D-Flip-Flop, effectively freezing the prescaler. Simultaneously, the final cnt_en output is masked via an AND gate (cnt_en = count_en & ~(halt_req & dbg_mode)), guaranteeing that no increments reach the main counter while halted.
- **Falling Edge Detector:** A D-Flip-Flop and an AND gate (with one inverted input) form a falling edge detector. It monitors timer_en and generates a 1-cycle timer_en_fall pulse when the timer is turned off. This pulse is routed to the Counter Module to trigger the hardware auto-clear function.

3.3.2. Timing and Behavior

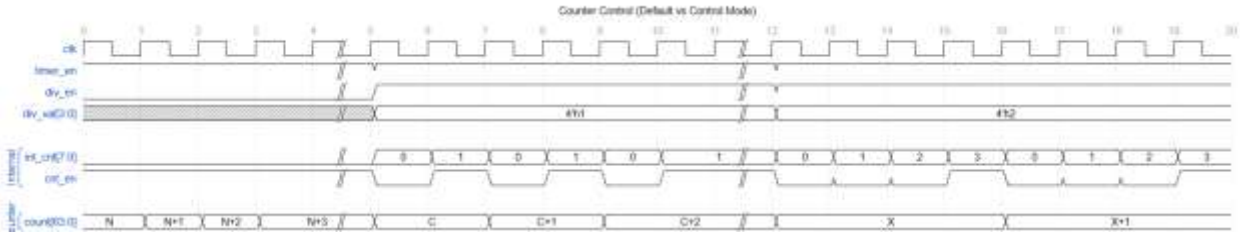


Figure 3.8: Counter Control Modes Timing

Figure 3.8 illustrates the dynamic behavior of the prescaler across different configurations.

- **Default Mode:** When div_en is Low, cnt_en remains persistently High, incrementing count[63:0] on every clock edge.
- **Divide-by-2 (div_val = 1):** The limit evaluates to 1. int_cnt toggles between 0 and 1, producing a cnt_en pulse exactly every 2 clock cycles.
- **Divide-by-4 (div_val = 2):** The limit evaluates to 3. int_cnt counts sequentially from 0 to 3, issuing a cnt_en pulse every 4 clock cycles, resulting in a proportionally slower counting rate for the main 64-bit register.

3.4. Counter Module

The Counter module is the core execution unit of the Timer IP, housing the physical 64-bit counting register (split into the 32-bit TDR0 and TDR1). It is strictly responsible for processing the counting arithmetic, facilitating byte-level overrides from the APB bus, and executing the hardware auto-clear mechanism without software intervention.

3.4.1. Logic Implementation and Operation Priority

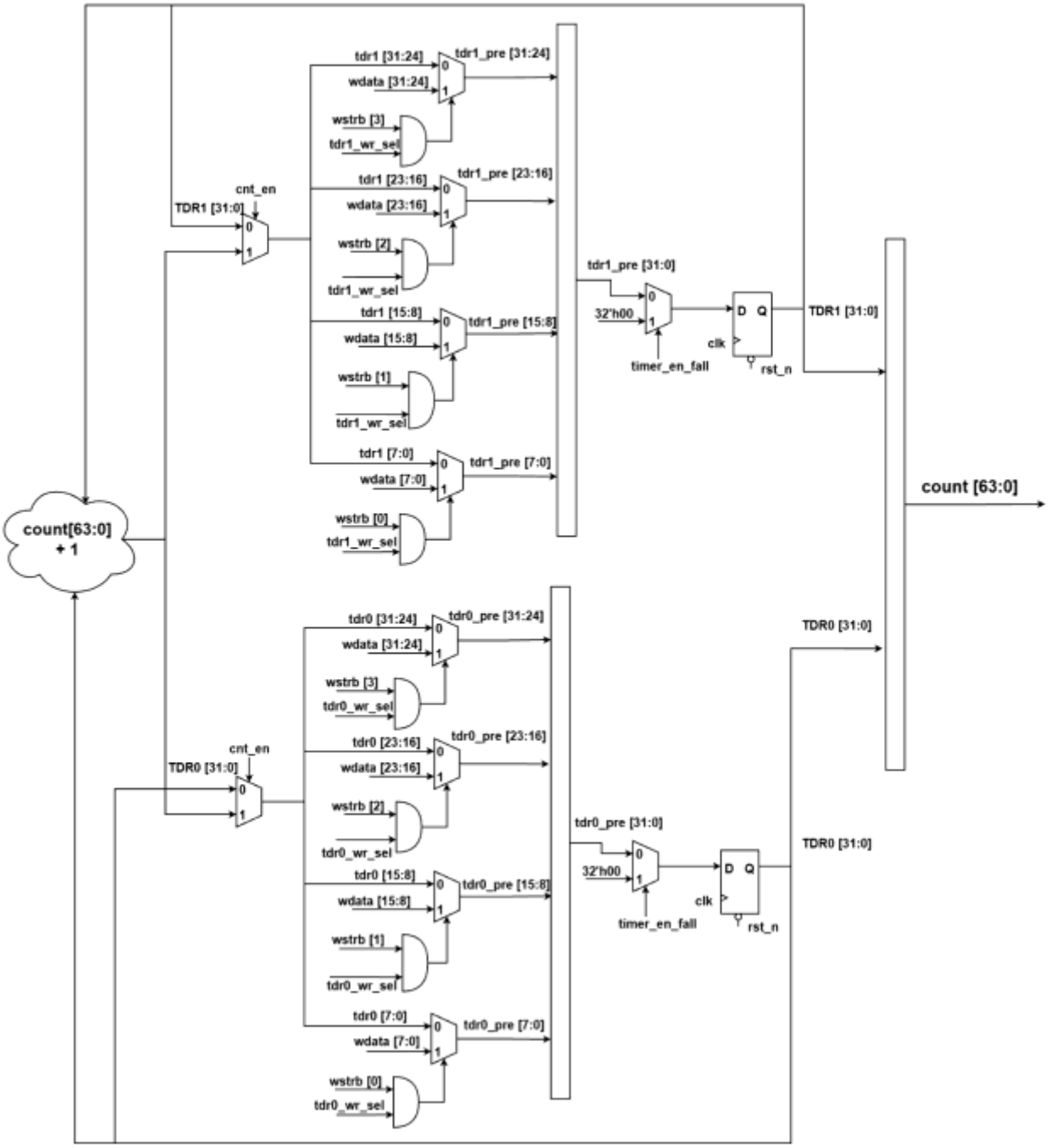


Figure 3.9: Counter Logic Diagram illustrating strict prioritization

As depicted in Figure 3.9, the data input for the TDR Flip-Flops is determined by a cascaded multiplexer architecture. This structural design establishes a strict mathematical hierarchy of operations to prevent conflicts:

1. **Hardware Auto-Clear (Highest Priority):** The final multiplexer directly preceding the D-Flip-Flops is controlled by the timer_en_fall pulse. If this pulse is asserted, the register is forced to load 32'h0000_0000 (32'h00 for both TDR0 and TDR1), completely overriding any ongoing counting or APB write operations.
2. **APB Byte Access (Medium Priority):** The preceding multiplexer handles software writes. The APB write selection (tdr0_wr_sel or tdr1_wr_sel) is combined with the byte strobe (wstrb[3:0]) using logical AND gates. This byte-enable architecture ensures that only the specific 8-bit segments targeted by the master are overwritten by wdata[31:0], while other unselected bytes safely retain their current values.
3. **Count Increment (Lowest Priority):** If neither a hardware clear nor an APB write is requested, the primary multiplexer decides whether to hold the current state or load the incremented value (count + 1). This decision is dynamically governed by the cnt_en pulse provided by the Counter Control module.

Both TDR0 and TDR1 dynamically concatenate to form the global count[63:0] bus, which is continuously fed back to the 64-bit adder and routed to the Register module for read access.

3.4.2. Advanced Features Timing

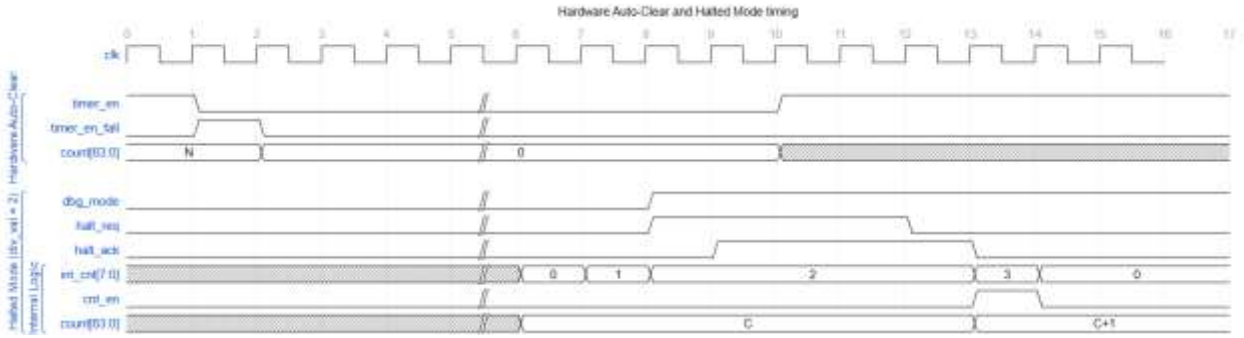


Figure 3.10: Hardware Auto-Clear and Halted Mode timing

Figure 3.10 illustrates the Advanced Level execution of the hardware auto-clear requirement.

- **Hardware Auto-Clear (Top Phase):** When the timer is explicitly disabled by software (timer_en transitions from High to Low), the falling edge detector in the Counter Control module generates a 1-cycle timer_en_fall pulse. On the immediately following clock edge, the count[63:0] bus synchronously drops to 0 (denoted by the value 0 on the bus), ensuring the timer is perfectly initialized and ready for its next usage cycle without requiring the CPU to manually write zeros to TDR0 and TDR1.
- **Halted Mode (Bottom Phase):** The lower portion of the waveform reiterates the Halted Mode timing. It visually confirms that when halt_req is asserted during debug mode, both the internal prescaler (int_cnt) and the main count[63:0] completely freeze their states, meticulously preserving the exact counting period once the halt request is lifted.

3.5. Interrupt Logic

The Interrupt logic block is responsible for continuously monitoring the timer's progress and generating a hardware-level interrupt when a predefined threshold is reached. It features a robust Write-1-to-Clear (RW1C) mechanism to ensure that software can safely acknowledge and clear pending interrupts without risking race conditions.

3.5.1. Logic Implementation and Clear Priority

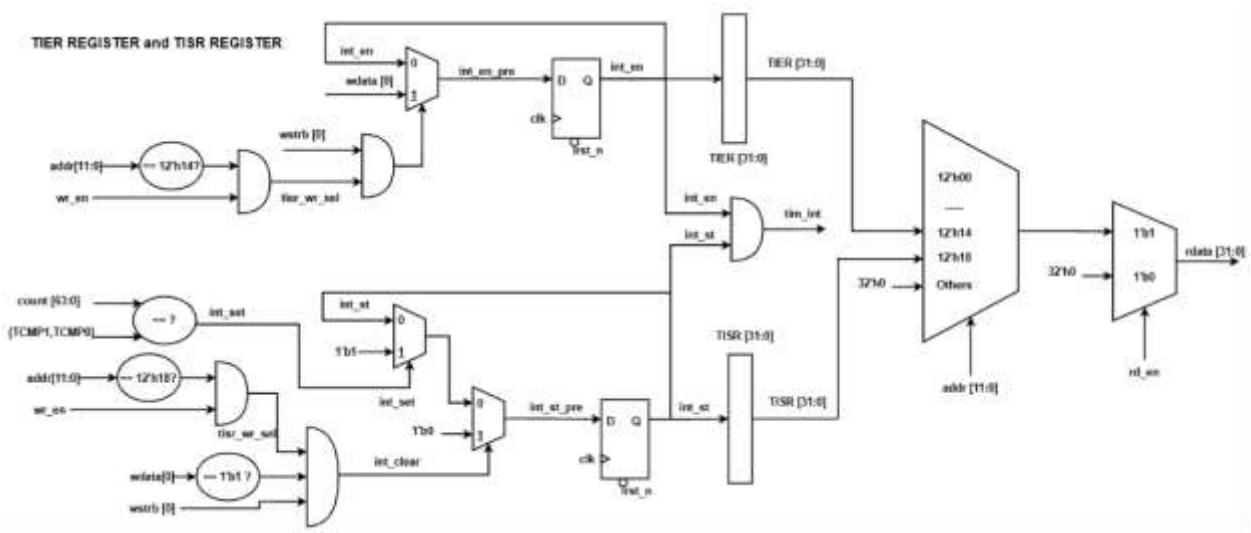


Figure 3.11: Interrupt Logic Diagram (Generation and RW1C Clearance)

As illustrated in Figure 3.11, the interrupt operation relies on hardware comparators and a prioritized multiplexer structure:

- **Interrupt Generation (int_set):** A 64-bit hardware comparator continuously checks the live count[63:0] bus against the concatenated {TCMP1, TCMP0} threshold. Upon an exact match, the combinatorial int_set signal is asserted. This signal requests the int_st (interrupt status/pending) Flip-Flop to transition to 1 on the next clock cycle.
- **Interrupt Masking:** The raw pending status (int_st) is routed through a logical AND gate alongside the int_en bit from the TIER register. The resulting signal, tim_int, is driven to the external system. This design allows the software to mask the physical interrupt output (tim_int = 0) while still preserving the internal pending state (int_st = 1) for polling purposes.
- **RW1C Clearance (int_clear):** To clear the pending interrupt, the APB Master must perform a write operation to the TISR address (12'h18) with the lowest bit set (wdata[0] == 1). An AND gate decodes this condition (addr == 12'h18 & wr_en & wstrb[0] & wdata[0]) to generate the int_clear pulse.

- **Hardware Priority:** Following the specification requirements, the clearance condition must have the highest priority. In the logic diagram (Figure 3.11), the multiplexer controlled by int_clear is positioned closest to the D-Flip-Flop's input, guaranteeing that if a software clear and a hardware set occur concurrently, the clear operation strictly overrides the set operation.

3.5.2. Interrupt Lifecycle Timing

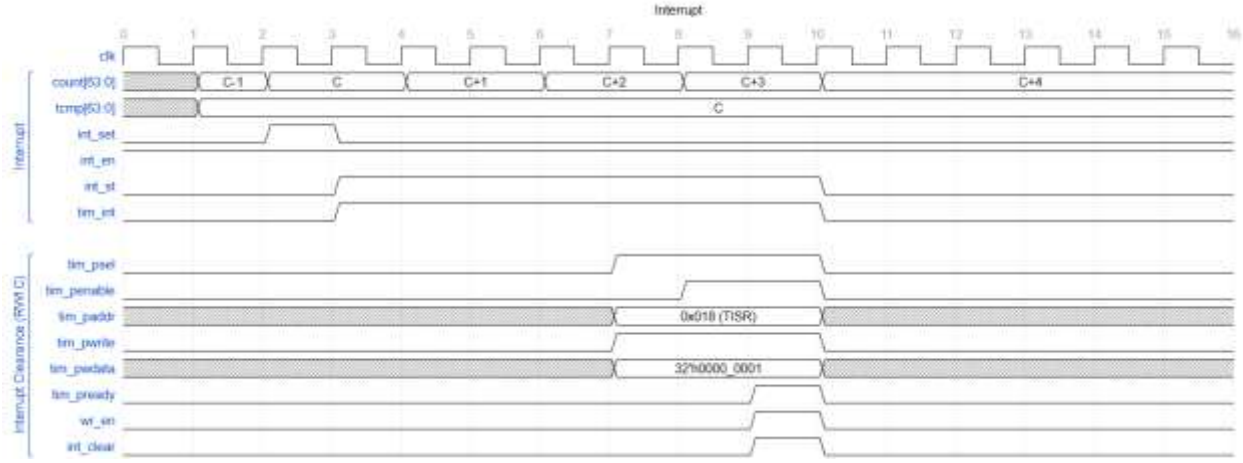


Figure 3.12: Interrupt Generation and RW1C Clearance timing

Figure 3.12 visually traces the complete lifecycle of a timer interrupt from hardware generation to software resolution:

1. **Assertion Phase (Left portion):** As the main counter increments, it eventually evaluates as strictly equal to tcmp[63:0] (denoted by C). The int_set pulse is immediately fired. On the subsequent clock rising edge, int_st and tim_int are synchronously asserted High. Notably, the count[63:0] continues its upward increment (C+1, C+2, etc.) without stalling, demonstrating that the interrupt generation does not interfere with the core timing operations.
2. **Clearance Phase (Right portion):** The APB Master initiates an active write transaction to address 0x018 (TISR) with a payload of 32'h0000_0001. Once the APB wait state resolves, the internal wr_en pulse is triggered, which in turn asserts the int_clear signal. On the next clock edge, the int_st pending bit and the tim_int

output are synchronously forced back to 0, successfully acknowledging and clearing the interrupt.